

Tecniche avanzate

Organizzazione del Codice

Come dividere un programma C++ in più file usando header file e file di implementazione.

Organizzare il codice in più file

Quando scrivi un programma piccolo, un unico file `.cpp` va benissimo. Ma immagina di dover scrivere un gestionale scolastico con centinaia di funzioni: tutto in un file solo sarebbe un disastro. Trovare una funzione richiederebbe minuti.

Dividere il codice in più file risolve questo problema. Ogni file ha uno scopo preciso, come i capitoli di un libro.

Vantaggi:

- **Leggibilità:** ogni file è focalizzato su un argomento
- **Riutilizzo:** puoi usare le stesse funzioni in più programmi
- **Manutenzione:** modifichi una parte senza toccare il resto
- **Collaborazione:** più persone lavorano su file diversi senza interferirsi

I due tipi di file

In C++ il codice viene diviso in due tipi di file:

Tipo	Estensione	Cosa contiene
Header	<code>.h</code> o <code>.hpp</code>	Le dichiarazioni (cosa esiste)
Implementazione	<code>.cpp</code>	Le definizioni (come funziona)

Il file header è come il **sommario di un libro**: ti dice cosa c'è, ma non ti mostra tutto il contenuto. Il file `.cpp` è il contenuto vero e proprio.

Esempio pratico

Immaginiamo di creare una piccola libreria con funzioni matematiche personalizzate.

Struttura dei file

```
progetto/  
├─ main.cpp  
├─ matematica.h  
└─ matematica.cpp
```

Il file header: **matematica.h**

Contiene solo le **firme** delle funzioni (senza il corpo):

```
// matematica.h – le dichiarazioni  
  
#ifndef MATEMATICA_H // protezione dall'inclusione multipla (spiegata più  
sotto)  
#define MATEMATICA_H  
  
int somma(int a, int b);  
int prodotto(int a, int b);  
double potenza(double base, int esponente);  
  
#endif
```

Il file di implementazione: **matematica.cpp**

Contiene il codice vero delle funzioni:

```
// matematica.cpp - le implementazioni

#include "matematica.h" // include il proprio header

int somma(int a, int b) {
    return a + b;
}

int prodotto(int a, int b) {
    return a * b;
}

double potenza(double base, int esponente) {
    double risultato = 1.0;
    for (int i = 0; i < esponente; i++) {
        risultato *= base;
    }
    return risultato;
}
```

Il file principale: `main.cpp`

Include l'header per sapere che le funzioni esistono:

```
// main.cpp
#include <iostream>
#include "matematica.h" // include il nostro header (virgolette, non <>)
using namespace std;

int main() {
    cout << "Somma: " << somma(3, 4) << endl;
    cout << "Prodotto: " << prodotto(3, 4) << endl;
    cout << "2^8 = " << potenza(2, 8) << endl;
    return 0;
}
```

Per compilare, passa **tutti** i file `.cpp` al compilatore:

```
g++ main.cpp matematica.cpp -o programma
./programma
```

Le include guard: protezione dall'inclusione multipla

Nell'header hai visto queste righe "strane":

```
#ifndef MATEMATICA_H
#define MATEMATICA_H

// ... contenuto ...

#endif
```

Si chiamano **include guard**. Servono a evitare un errore comune: se lo stesso header venisse incluso due volte nello stesso file, il compilatore si lamenterebbe di funzioni "ridefinite".

Come funzionano:

1. `#ifndef MATEMATICA_H` — "se MATEMATICA_H non è ancora definito..."
2. `#define MATEMATICA_H` — "...definiscilo..."
3. "...ed esegui tutto questo contenuto"
4. `#endif` — fine del blocco

La seconda volta che il file viene incluso, `MATEMATICA_H` è già definito, quindi tutto il contenuto viene saltato automaticamente.

Alternativa moderna: `#pragma once`

```
#pragma once // includi questo file una sola volta

// contenuto dell'header...
```

`#pragma once` fa la stessa cosa delle include guard, ma è più semplice da scrivere. È supportata da tutti i compilatori moderni.

Come includere file

Ci sono due sintassi per `#include` :

```
#include <iostream>    // parentesi angolari → librerie di sistema
#include "matematica.h" // virgolette → file del tuo progetto
```

Esempio con una classe

Le classi si dividono esattamente nello stesso modo: la dichiarazione nell'header, i metodi nel `.cpp` .

studente.h

```
#pragma once

#include <string>
using namespace std;

class Studente {
private:
    string nome;
    int eta;
    double votoMedio;

public:
    Studente(string n, int e);           // solo le firme dei metodi
    void aggiungiVoto(double voto);
    void stampaInfo() const;
    string getNome() const;
    double getVotoMedio() const;
};
```

studente.cpp

```
#include "studente.h"
#include <iostream>
using namespace std;

// L'implementazione dei metodi usa NomeClasse:: davanti al nome del metodo
Studente::Studente(string n, int e) {
    nome = n;
    eta = e;
    votoMedio = 0.0;
}

void Studente::aggiungiVoto(double voto) {
    if (votoMedio == 0.0) {
        votoMedio = voto;
    } else {
        votoMedio = (votoMedio + voto) / 2.0;
    }
}

void Studente::stampaInfo() const {
    cout << "Nome: " << nome << ", Eta: " << eta
        << ", Media: " << votoMedio << endl;
}

string Studente::getNome() const { return nome; }
double Studente::getVotoMedio() const { return votoMedio; }
```

main.cpp

```
#include <iostream>
#include "studente.h"
using namespace std;

int main() {
    Studente s("Mario", 16);
    s.aggiungiVoto(8.5);
    s.aggiungiVoto(7.0);
    s.aggiungiVoto(9.0);
    s.stampaInfo();
    return 0;
}
```

Compilazione:

```
g++ main.cpp studente.cpp -o programma
```

Cosa va dove

Nell'header (`.h`) metti:

- Le firme delle funzioni (prototipi)
- Le dichiarazioni delle classi
- Le definizioni di `struct` ed `enum`
- Le costanti (`const`)

Nel file `.cpp` metti:

- Il corpo delle funzioni
- Il corpo dei metodi delle classi
- La logica del programma

Regola pratica: se qualcuno ha bisogno di *sapere che esiste*, mettilo nell'header. Se qualcuno ha bisogno di *vedere come funziona*, mettilo nel `.cpp`.

Errori comuni

```
// Dimenticare le include guard → errori di ridefinizione
// Soluzione: aggiungi #pragma once o #ifndef

// Includere il .cpp invece dell'header
#include "matematica.cpp" // SBAGLIATO
#include "matematica.h"   // CORRETTO

// Dimenticare di compilare tutti i file .cpp
g++ main.cpp -o programma // SBAGLIATO — manca matematica.cpp
g++ main.cpp matematica.cpp -o programma // CORRETTO
```

Compilazione e Linking

Come funziona il processo di trasformazione del codice C++ in un programma eseguibile.

Come funziona la compilazione?

Quando premi "compila" nel tuo editor, sembra che il programma parta magicamente. In realtà, dietro le quinte succedono tre cose distinte.

Capire questo processo ti aiuta a:

- **interpretare gli errori** — ci sono errori di compilazione ed errori di linking, e si risolvono in modi diversi
- **lavorare con progetti grandi** — con più file `.cpp`, devi sapere come metterli insieme
- **usare il terminale** — spesso si compila da riga di comando, specialmente in ambito professionale

Le tre fasi: dalla ricetta al piatto

Pensa al tuo codice come a una ricetta scritta in italiano. Per "eseguirlo" su un computer, bisogna:

1. **Preprocessing** — il preprocessore prepara gli ingredienti (espande `#include`, sostituisce le macro `#define`, ecc.)
2. **Compilazione** — il compilatore traduce la ricetta in lingua macchina

3. **Linking** — il linker unisce tutti i pezzi in un unico programma eseguibile

Fase 1: Preprocessing

Il preprocessore elabora il tuo codice

prima

della compilazione. Sostituisce ogni `#include` con il contenuto del file incluso, espande le macro e gestisce i blocchi `#ifdef`.

```
#include <iostream>    // viene sostituito con tutto il contenuto di
                        iostream
#define PI 3.14159     // ogni occorrenza di PI viene sostituita con 3.14159

int main() {
    // dopo il preprocessing questa riga diventa:
    // double area = 3.14159 * 5 * 5;
    double area = PI * 5 * 5;
    return 0;
}
```

Puoi vedere il risultato del preprocessing con il flag `-E` :

```
g++ -E main.cpp -o main.i
```

Il file `main.i` contiene il codice dopo il preprocessing — di solito è molto lungo perché include tutto il contenuto degli header.

Fase 2: Compilazione

Il compilatore prende il codice preprocessato e lo trasforma in **codice oggetto** — un file in linguaggio macchina con estensione `.o` (su Linux e macOS) o `.obj` (su Windows).

Il codice oggetto contiene istruzioni che il processore capisce, ma non è ancora un programma completo: mancano i collegamenti alle funzioni scritte in altri file.

```
# Compila senza collegare (-c = solo compilazione)
g++ -c main.cpp -o main.o
g++ -c matematica.cpp -o matematica.o
```

Fase 3: Linking (collegamento)

Il linker prende tutti i file oggetto e li **unisce** in un unico programma eseguibile. Risolve i "ponti" tra i file: se `main.cpp` chiama la funzione `somma()` definita in `matematica.cpp`, il linker collega le due parti.

```
# Collega i file oggetto in un unico eseguibile
g++ main.o matematica.o -o programma
```

Il comando completo con g++

Di solito fai tutto in un unico comando, senza pensare alle fasi:

```
g++ main.cpp matematica.cpp -o programma
```

Questo comando esegue automaticamente tutte e tre le fasi per ogni file `.cpp` e poi collega tutto insieme.

I flag di g++ più utili

I **flag** sono opzioni che puoi aggiungere al comando per modificarne il comportamento. Iniziano sempre con `-`.

Il nome dell'output

```
g++ file.cpp -o mio_programma # l'eseguibile si chiamerà "mio_programma"
g++ file.cpp                  # senza -o, l'eseguibile si chiama "a.out"
```

Warning — gli avvisi del compilatore

I warning non sono errori: il programma compila lo stesso, ma il compilatore ti avvisa di cose sospette.

```
g++ -Wall file.cpp -o output      # abilita la maggior parte dei warning
g++ -Wextra file.cpp -o output    # warning aggiuntivi
g++ -Wall -Wextra file.cpp -o output # entrambi – consigliato!
```

Esempi di warning utili:

```
int x;           // warning: x usata senza essere inizializzata
if (x = 5)        // warning: probabilmente volevi == invece di =
```

Leggi sempre i warning e correggili — anche se il programma funziona.

Debug — per trovare gli errori

```
g++ -g file.cpp -o output # include informazioni di debug nel programma
```

Con `-g` puoi usare strumenti come `gdb` per eseguire il programma passo passo e vedere cosa succede.

Ottimizzazione — per rendere il programma più veloce

```
g++ -O0 file.cpp -o output # nessuna ottimizzazione (utile durante lo sviluppo)
g++ -O1 file.cpp -o output # ottimizzazione leggera
g++ -O2 file.cpp -o output # ottimizzazione moderata (consigliata per la versione finale)
g++ -O3 file.cpp -o output # ottimizzazione aggressiva
```

Usa `-O0` durante lo sviluppo e il debug. Usa `-O2` quando vuoi distribuire il programma.

Standard C++ — scegliere la versione del linguaggio

```
g++ -std=c++11 file.cpp -o output # C++11
g++ -std=c++14 file.cpp -o output # C++14
g++ -std=c++17 file.cpp -o output # C++17 (buona scelta moderna)
g++ -std=c++20 file.cpp -o output # C++20
```

Specifica sempre lo standard se usi funzionalità moderne. C++17 è una buona scelta oggi:

```
g++ -std=c++17 -Wall -Wextra main.cpp -o programma
```

Aggiungere cartelle per gli header

```
g++ -I./include file.cpp -o output # cerca gli header anche nella cartella "include"
g++ -I /percorso/assoluto file.cpp -o output
```

Errori di compilazione vs errori di linking

Saper distinguere i due tipi di errore ti fa trovare il problema molto più in fretta.

Errori di compilazione

Riguardano la **sintassi** o la **logica** del tuo codice. Il compilatore non capisce cosa hai scritto:

```
main.cpp:5:10: error: 'somma' was not declared in this scope
```

Hai usato `somma` senza dichiararla. Manca il prototipo della funzione o l' `#include` dell'header giusto.

```
main.cpp:8:5: error: expected ';' before '}' token
```

Hai dimenticato un punto e virgola.

Errori di linking

Riguardano **riferimenti non risolti** tra file diversi. Il compilatore conosce la funzione, ma il linker non trova la sua definizione:

```
undefined reference to `somma(int, int)'
```

Significa: hai dichiarato `somma` (il compilatore è felice), ma non hai dato al linker il file `.cpp` che la definisce.

Soluzione: aggiungi il file mancante al comando:

```
# SBAGLIATO: manca matematica.cpp
g++ main.cpp -o programma

# CORRETTO
g++ main.cpp matematica.cpp -o programma
```

Esempio pratico con più file

Struttura del progetto:

```
progetto/  
├─ main.cpp  
├─ studente.h  
└─ studente.cpp
```

Compilazione in un unico comando:

```
g++ -std=c++17 -Wall main.cpp studente.cpp -o programma
```

Oppure in due fasi (utile nei progetti grandi — ricompili solo i file modificati):

```
# Compila i singoli file  
g++ -std=c++17 -c main.cpp -o main.o  
g++ -std=c++17 -c studente.cpp -o studente.o  
  
# Collega tutto  
g++ main.o studente.o -o programma
```

Makefile — automatizzare la compilazione

Nei progetti con molti file, riscrivere il comando `g++` ogni volta è noioso e rischioso. Il **Makefile** è un file che descrive come compilare il progetto automaticamente.

```
# Makefile
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra

programma: main.o studente.o
    $(CXX) main.o studente.o -o programma

main.o: main.cpp studente.h
    $(CXX) $(CXXFLAGS) -c main.cpp

studente.o: studente.cpp studente.h
    $(CXX) $(CXXFLAGS) -c studente.cpp

clean:
    rm -f *.o programma
```

Per compilare, scrivi solo:

```
make
```

Per cancellare i file compilati:

```
make clean
```

Non devi padroneggiare il Makefile subito — ma è utile sapere che esiste.

Riepilogo dei flag principali

Flag	Cosa fa
<code>-o nome</code>	Specifica il nome del file di output
<code>-Wall</code>	Abilita i warning principali
<code>-Wextra</code>	Warning aggiuntivi
<code>-g</code>	Aggiunge informazioni per il debug
<code>-O2</code>	Ottimizzazione (consigliata per la versione finale)
<code>-std=c++17</code>	Usa lo standard C++17
<code>-c</code>	Solo compilazione, senza collegamento
<code>-I percorso</code>	Aggiunge una cartella dove cercare gli header
<code>-E</code>	Solo preprocessing

Comando consigliato durante lo studio

Usa sempre questo comando quando studi e sviluppi:

```
g++ -std=c++17 -Wall -Wextra file.cpp -o programma
```

I warning sono tuoi alleati: leggili con attenzione e correggili sempre, anche quando il programma compila senza errori.

Namespace

Cosa sono i namespace e come si usa std in C++.

Cos'è un namespace?

Immagina che due studenti nella stessa classe si chiamino entrambi "Marco". Ogni volta che la prof dice "Marco!", nasce confusione. La soluzione? Usare il cognome: "Marco Rossi" e "Marco Bianchi".

I **namespace** in C++ funzionano allo stesso modo. Se due librerie diverse definiscono una funzione chiamata `print`, il compilatore non saprebbe quale usare. Con i namespace, puoi distinguerle: `LibreriaA::print` e `LibreriaB::print`.

Il namespace `std`

Tutto quello che fa parte della libreria standard del C++ si trova nel namespace `std` (abbreviazione di "standard"). Per usare `cout`, devi indicare che appartiene a `std`:

```
std::cout << "Ciao!" << std::endl;
std::string nome = "Alice";
```

Il `::` si legge "appartenente a". Quindi `std::cout` significa "cout appartenente a std".

using namespace std : la scorciatoia

Scrivere `std::` ogni volta è scomodo. Con `using namespace std;` importi tutto il namespace `std` e puoi usarlo senza prefisso:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Ciao!" << endl; // invece di std::cout e std::endl
    string nome = "Alice";   // invece di std::string
    return 0;
}
```

Questa è la forma che hai visto in quasi tutto il codice di questo manuale.

using per nomi specifici

Puoi importare solo i nomi che ti servono, invece di tutto il namespace:

```
#include <iostream>
using std::cout;
using std::endl;
using std::string;

int main() {
    cout << "Ciao!" << endl;    // OK - importato
    // cin >> x;                 // ERRORE - cin non è stato importato
    return 0;
}
```

Questo è più preciso ed è la scelta preferita nei progetti grandi.

Creare i tuoi namespace

Puoi usare i namespace per organizzare il tuo codice ed evitare conflitti tra funzioni con lo stesso nome:

```

namespace Matematica {
    const double PI = 3.14159265358979;

    double area(double raggio) {
        return PI * raggio * raggio;    // area del cerchio
    }

    double perimetro(double raggio) {
        return 2 * PI * raggio;
    }
}

namespace Geometria {
    double area(double base, double altezza) {
        return base * altezza;    // area del rettangolo
    }
}

int main() {
    // Nessun conflitto – il compilatore sa qual è quale
    cout << Matematica::area(5.0) << endl;    // area del cerchio
    cout << Geometria::area(4.0, 6.0) << endl; // area del rettangolo
    cout << Matematica::PI << endl;
    return 0;
}

```

Namespace annidati

Puoi mettere namespace dentro altri namespace, come cartelle dentro cartelle:

```

namespace Scuola {
    namespace Matematica {
        double calcolaMedia(double a, double b) {
            return (a + b) / 2;
        }
    }

    namespace Italiano {
        string maiuscolo(string s) {
            return s; // semplificato
        }
    }
}

int main() {
    cout << Scuola::Matematica::calcolaMedia(8, 9) << endl; // 8.5
    return 0;
}

```

In C++17 puoi scrivere i namespace annidati in modo più compatto:

```

namespace Scuola::Matematica {
    double calcolaMedia(double a, double b) {
        return (a + b) / 2;
    }
}

```

Alias per namespace lunghi

Se un namespace ha un nome lungo, puoi dargli un soprannome:

```

namespace MioSistemaGestioneStudenti {
    void aggiungiStudiante(string nome) { /* ... */ }
}

namespace MSGS = MioSistemaGestioneStudenti;    // soprannome

int main() {
    MSGS::aggiungiStudiante("Alice");    // molto più comodo!
    return 0;
}

```

Regola importante per gli header

Non scrivere mai `using namespace std;` dentro un file header (`.h`). Chi include il tuo header si ritroverebbe importato tutto `std` senza saperlo — potrebbe causare conflitti di nomi inaspettati:

```

// mio_header.h – SBAGLIATO
using namespace std;    // "inquina" il namespace di chi include questo file

// mio_header.h – CORRETTO
void funzione(std::string s);    // usa il prefisso esplicito

```

Esempio pratico: calcolo geometrico

```
#include <iostream>
#include <cmath>
using namespace std;

namespace Cerchio {
    const double PI = 3.14159265358979;
    double area(double r) { return PI * r * r; }
    double perimetro(double r) { return 2 * PI * r; }
}

namespace Rettangolo {
    double area(double b, double a) { return b * a; }
    double perimetro(double b, double a) { return 2 * (b + a); }
    double diagonale(double b, double a) { return sqrt(b*b + a*a); }
}

int main() {
    double r = 5.0;
    cout << "Cerchio con raggio " << r << ":" << endl;
    cout << "  Area: " << Cerchio::area(r) << endl;
    cout << "  Perimetro: " << Cerchio::perimetro(r) << endl;

    double b = 4.0, a = 3.0;
    cout << "\nRettangolo " << b << "x" << a << ":" << endl;
    cout << "  Area: " << Rettangolo::area(b, a) << endl;
    cout << "  Perimetro: " << Rettangolo::perimetro(b, a) << endl;
    cout << "  Diagonale: " << Rettangolo::diagonale(b, a) << endl;

    return 0;
}
```

Preprocessore

Le direttive del preprocessore in C++, come

Cos'è il preprocessore?

Quando scrivi un programma in C++, prima ancora che il compilatore legga il tuo codice, un altro programma — chiamato **preprocessore** — fa una "pulizia e preparazione" del testo.

Immagina di consegnare un compito scritto a matita a un assistente: lui prima cancella i riferimenti, sostituisce alcune parole con quelle giuste, aggiunge i fogli mancanti... e solo dopo lo consegna al professore (il compilatore) per la correzione.

Le istruzioni che dai al preprocessore si chiamano **direttive** e iniziano sempre con il simbolo `#`.

`#include` — Aggiungere altri file

`#include` dice al preprocessore: *"Copia il contenuto di questo file e mettilo qui."*

È come dire: "Aggiungi questo capitolo del libro al mio testo."

Librerie del sistema

Per includere le librerie già pronte che vengono con C++, si usano le parentesi angolari `<>`:

```
#include <iostream>    // per stampare e leggere dati
#include <string>       // per usare le stringhe
#include <cmath>        // per le funzioni matematiche
#include <vector>       // per usare i vettori
```

Il compilatore sa già dove trovare questi file nel tuo computer.

File tuoi

Per includere file che hai scritto tu, si usano le virgolette `""`:

```
#include "studente.h"    // file nella stessa cartella
#include "modelli/persona.h" // file in una sottocartella
```

Il compilatore cerca prima nella cartella del tuo progetto, poi nelle cartelle standard.

#define — Creare una sostituzione automatica

`#define` è come creare una scorciatoia: dici al preprocessore "ogni volta che vedi questa parola, sostituiscila con quest'altra".

Costanti con `#define`

```
#define PI 3.14159265358979
#define MAX_STUDENTI 30
#define SALUTO "Ciao, mondo!"

int main() {
    // Il preprocessore sostituisce PI prima ancora di compilare:
    // double area = 3.14159265358979 * 5.0 * 5.0;
    double area = PI * 5.0 * 5.0;
    return 0;
}
```

Oggi si preferisce usare `const` al posto di `#define` per le costanti, perché è più sicuro. Ma `#define` si vede spesso nel codice esistente.

Macro con parametri (scorciatoie con calcoli)

Puoi creare scorciatoie che accettano valori:

```
#define MASSIMO(a, b) ((a) > (b) ? (a) : (b))
#define QUADRATO(x) ((x) * (x))

int main() {
    int x = 5, y = 3;
    cout << MASSIMO(x, y) << endl;    // stampa 5
    cout << QUADRATO(4) << endl;     // stampa 16
    return 0;
}
```

Attenzione: queste "macro-funzioni" possono dare risultati strani. Preferisci le funzioni normali:


```
// PROBLEMA: n viene incrementato due volte!
int n = 5;
int r = QUADRATO(n++); // diventa: ((n++) * (n++)) – comportamento
                           imprevedibile

// SOLUZIONE: usa una funzione normale
int quadrato(int x) { return x * x; }
```

#undef — Cancellare una scorciatoia

```
#define VALORE 42
cout << VALORE << endl; // 42

#undef VALORE
// Da qui in poi VALORE non esiste più
// cout << VALORE << endl; // ERRORE!
```

Compilazione condizionale — Includere codice solo in certi casi

Puoi dire al preprocessore:

"includi questa parte di codice solo se questa condizione è vera"

.

È come avere due versioni di un documento e scegliere quale stampare:

```
#define DEBUG    // commenta questa riga per disabilitare i messaggi di
debug

int main() {
    int x = 42;

#ifdef DEBUG
    // Questa riga viene inclusa SOLO se DEBUG è definito
    cout << "Debug: x = " << x << endl;
#endif

    cout << "Valore: " << x << endl;
    return 0;
}
```

- `#ifdef NOME` = "se NOME è definito, includi il codice che segue"
- `#ifndef NOME` = "se NOME NON è definito, includi il codice che segue"
- `#endif` = "fine del blocco condizionale"

Include guard — Evitare di includere lo stesso file due volte

Immagina di copiare e incollare lo stesso capitolo due volte in un libro: sarebbe un problema. Stessa cosa in C++.

Quando scrivi un file `.h`, usa sempre questa protezione:

```
// studente.h
#ifndef STUDENTE_H    // se STUDENTE_H non è ancora definito...
#define STUDENTE_H    // ...definiscilo (ed esegui il codice qui sotto)

// Tutto il contenuto del file va qui
void salutaStudiante();
struct Studente { int eta; };

#endif    // fine della protezione
```

La seconda volta che questo file viene incluso (magari attraverso un altro file), `STUDENTE_H` è già definito, quindi tutto il contenuto viene saltato automaticamente.

In alternativa, puoi usare `#pragma once`, che fa la stessa cosa in modo più semplice:

```
// studente.h
#pragma once // includi questo file al massimo una volta

void salutaStudente();
```

`#pragma once` è supportato da quasi tutti i compilatori moderni, anche se non è parte dello standard ufficiale.

#error — Generare un errore personalizzato

Puoi far fallire la compilazione con un messaggio tuo:

```
#ifndef MIA_LIBRERIA_H
#error "Devi includere mia_libreria.h prima di questo file!"
#endif
```

Se la condizione si verifica, il compilatore si ferma e mostra il tuo messaggio.

Macro predefinite — Informazioni automatiche

Il preprocessore definisce alcune parole speciali in automatico:

```
cout << __FILE__ << endl; // nome del file corrente
cout << __LINE__ << endl; // numero della riga corrente
cout << __DATE__ << endl; // data in cui è stato compilato il programma
cout << __TIME__ << endl; // ora in cui è stato compilato
```

Sono molto utili per il debug — per sapere esattamente dove è successo un problema:

```
#ifndef DEBUG
// Crea un messaggio di log con il nome del file e il numero di riga
#define LOG(msg) cout << "[" << __FILE__ << ":" << __LINE__ << "]" " << msg
<< endl
#else
#define LOG(msg)    // in modalità normale, LOG non fa nulla
#endif

int main() {
    int x = 42;
    LOG("Il valore di x è: " << x);    // messaggio visibile solo in
    modalità debug
    return 0;
}
```